f5-staging-precheck

Documentation Handout — check_multi 2.0 & bigip-precheck

Generated 2026-08-10

Contents

1. check_multi 2.0 — Overview

check_multi 2.0

Enterprise Device Audit Framework

Multi-function, parallel SSH-based audit tool for network appliances (primarily F5 BIG-IP).

Documentation site: https://willyd61.github.io/f5-staging-precheck/

Overview

`check_multi` is a modular, secure, and governance-ready framework for running configuration, health, and compliance checks against large inventories of network devices.

It was completely redesigned from a legacy monolithic Bash script into a clean, extensible architecture following industry best practices.

> **New:** `bigip-precheck` is a Python, **iControl REST**–driven companion focused on **pre-upgrade / change-window readiness** for LTM & GTM. Where `check_multi` runs fast SSH/tmsh sweeps, `bigip-precheck` performs deeper REST checks and emits a per-device and overall **GO / NO-GO** verdict with a full audit trail. See its README and architecture. *(alpha; phase A)*

Key Features (Alpha)

Feature	Status
Modular library architecture	
External YAML configuration	
Plugin-style check modules	
Secure SSH abstraction	
Dry-run mode	
Structured logging + JSON	
Retry with backoff	
Inventory pre-flight validation	
Executive Excel reports	
Audit trail / run logging	
Environment profiles (lab/prod)	
Argument validation + safety prompts	
"Did you mean…?" check suggestions	

Feature	Status
Bash tab-completion	
GitHub Actions CI (lint, tests, smoke)	
Published docs site (GitHub Pages)	

Requirements

Tool	Purpose	Required
`bash` ≥ 4	Runtime	Yes
`ssh` / OpenSSH	Device access	Yes
`yq` (mikefarah)	YAML config/profiles	Optional (falls back to built-in defaults)
`python3` + `openpyxl`	`--excel` reports	Optional
`sshpass`	Password auth (`-p`, discouraged)	Optional
`shellcheck`, `bats`	Development / CI	Dev only

Quick Start

```
# Clone the repository
git clone https://github.com/willyd61/f5-staging-precheck.git
cd f5-staging-precheck

# View help
./bin/check_multi --help
```

```
# Safe dry-run against example inventory
./bin/check_multi --dry-run platform examples/devices.txt

# Real run with Excel report
./bin/check_multi -e lab -t 10 --excel platform examples/devices.txt
```

Documentation

The full, styled documentation is published to **GitHub Pages** and built from the Markdown under `docs/`:

Document	Description
QUICKSTART.md	First-run guide
INSTALL.md	Detailed installation instructions
USAGE.md	Full CLI reference and safety behaviours
CONFIGURATION.md	YAML config, profiles, env vars
MODULES.md	Check catalogue and roadmap
ARCHITECTURE.md	Technical design
SECURITY.md	Security model and recommendations
TROUBLESHOOTING.md	Common issues and fixes
FAQ.md	Frequently asked questions

Architecture

```
check_multi_2.0/
├── bin/check_multi              # Main CLI entry point
├── lib/
│   ├── common.sh                # Logging, colours, utils, temp files
│   ├── ssh.sh                   # Secure SSH + retry abstraction
│   ├── inventory.sh             # Device list loading & validation
│   ├── parallel.sh              # Concurrency control
│   ├── config.sh                # YAML config loader
│   ├── audit.sh                 # Run audit trail
│   ├── summary.sh               # summary.json generator
│   ├── registry.sh              # Check discovery + name validation
│   └── checks/                  # Plugin check modules
│       ├── platform.sh  umm.sh  certs.sh  keys.sh
│       ├── syncgroup.sh network.sh preflight.sh
│       └── ntp.sh  license.sh  diskspace.sh  ha.sh
├── config/
│   ├── defaults.yaml
│   └── environments/{lab,prod}.yaml
├── completions/check_multi.bash   # Bash tab-completion
```

```
├── scripts/
│   ├── generate_excel_report.py
│   ├── build_docs.py            # Static docs-site generator → site/
│   └── shellcheck_all.sh
├── docs/                        # Markdown → published to GitHub Pages
├── examples/  tests/  Makefile
└── .github/workflows/{ci.yml,docs.yml}
```

Usage

```
check_multi [OPTIONS] <check_name> <inventory_file>
```

Global Options

Option	Description
`-h, --help`	Show help
`-d, --dry-run`	No SSH connections
`-v, --verbose`	Debug logging
`-t, --threads <N>`	Max parallel sessions
`-o, --output-dir <DIR>`	Results directory
`-e, --env <lab|prod>`	Environment profile
`-c, --config <FILE>`	Override defaults.yaml
`-u, --user <USER>`	SSH username

Option	Description
`-p, --password`	Use password auth (discouraged)
`-y, --yes`	Assume "yes" for confirmation prompts
`--excel`	Generate executive Excel report
`--no-color`	Disable coloured output (also `NO_COLOR`)
`--list-checks`	List available check modules and exit
`--version`	Print version

Command-line flags always override YAML config. Real runs against `prod` or large inventories (≥ 50 devices) prompt for confirmation; use `-y` for unattended execution. See USAGE.md for full details.

Available Checks

Run `check_multi --list-checks` for the live list.

- `preflight` – Lightweight connectivity + version check
- `platform` – Version, build, modules, serial, telemetry
- `umm` – Provisioned modules
- `certs` – Certificate inventory
- `keys` – Key inventory
- `syncgroup` – CM sync status
- `network` – Interfaces & trunks
- `ntp` – NTP config and clock synchronisation
- `license` – Licence status and feature modules
- `diskspace` – Filesystem utilisation
- `ha` – Active/standby and failover state

New checks are added by dropping a module in `lib/checks/` — see MODULES.md.

Shell completion

```
# Enable tab-completion for the current shell
```

```
source completions/check_multi.bash
```

Security Model

1. **Prefer SSH keys** – password mode (`-p`) prints a large warning.
2. Temporary files are created with mode `0600` and removed on exit.
3. Output directories use `0750`.
4. Host key checking is controlled via environment profile.
5. Full audit trail of every run.

Development

All developer tasks are wrapped in the `Makefile`:

```
make lint        # ShellCheck every script (including bin/check_multi)

make syntax      # bash -n syntax check

make test        # bats unit tests

make smoke       # dry-run every check module (auto-discovered)

make check       # all of the above

make docs        # build the documentation site into ./site

make docs-serve  # build + serve docs at http://localhost:8000

make clean       # remove ./results and ./site
```

CI runs the same checks on every push and pull request (see `.github/workflows/ci.yml`), and the documentation site is built and deployed to GitHub Pages on merge to `main` (see `.github/workflows/docs.yml`). Please run `make check` before opening a PR. Contribution guidelines live in CONTRIBUTING.md; notable changes are recorded in CHANGELOG.md.

Colour output is emitted only to a TTY and honours the `NO_COLOR` convention.

License

Internal Use Only – Platform Engineering / Network Automation

---

check_multi 2.0.0-alpha

2. Quick Start

check_multi 2.0 Alpha – Quick Start

1. Prerequisites

- bash 4.2+
- ssh
- python3 + openpyxl (only if using --excel)
- yq (optional but recommended for YAML config)

2. First Run (safe)

```
./bin/check_multi --dry-run platform examples/devices.txt
```

3. Real Run

```
./bin/check_multi -e lab -t 10 platform examples/devices.txt
```

4. Executive Excel Report

```
./bin/check_multi -e prod --excel -t 15 certs inventory/prod.txt
```

The Excel file appears in the results directory and is suitable for emailing to leadership.

5. Available Checks

platform | umm | certs | keys | syncgroup | network | preflight

6. Useful Options

Flag	Purpose
`--dry-run`	Preview without SSH
`-t 15`	15 parallel sessions
`-e prod`	Use production profile
`--excel`	Generate VP-ready Excel report
`-v`	Verbose / debug logging

3. Installation

check_multi 2.0 Alpha – Installation Guide

1. Prerequisites

Component	Required	Notes	Typical Install Command
bash ≥ 4.2	Yes	Most modern Linux/macOS systems	pre-installed
OpenSSH client	Yes	`ssh` command	`openssh-client` / `openssh`
python3 ≥ 3.8	Optional	Only needed for `--excel` reports	`python3`
openpyxl	Optional	Excel report generation	`pip install openpyxl`
yq	Recommended	YAML config parsing	See below

Component	Required	Notes	Typical Install Command
shellcheck	Dev only	Static analysis	`shellcheck`
bats	Dev only	Test framework	`bats`
sshpass	Optional	Only if you must use password auth	`sshpass` (discouraged)

Installing yq (recommended)

```
# Linux (amd64)
sudo wget -qO /usr/local/bin/yq \
  https://github.com/mikefarah/yq/releases/latest/download/yq_linux_amd64
sudo chmod +x /usr/local/bin/yq

# macOS (Homebrew)
brew install yq
```

Installing openpyxl (for Excel reports)

```
python3 -m pip install --user openpyxl
# or inside a virtualenv
python3 -m venv .venv
source .venv/bin/activate
pip install openpyxl
```

---

2. Installation Methods

Method A – Git Clone (Recommended)

```
git clone https://github.com/<your-org>/check_multi.git
cd check_multi

chmod 0755 bin/check_multi
export PATH="$PWD/bin:$PATH"
check_multi --version
```

Method B – Release Tarball / Zip

```
tar xzf check_multi-2.0.0-alpha.tar.gz
cd check_multi_2.0
chmod 0755 bin/check_multi
./bin/check_multi --version
```

Method C – User-local install (no root)

```
mkdir -p ~/bin ~/opt
cp -a check_multi_2.0 ~/opt/check_multi
ln -sf ~/opt/check_multi/bin/check_multi ~/bin/check_multi
export PATH="$HOME/bin:$PATH"
check_multi --version
```

Method D – System-wide install (requires root)

```
sudo mkdir -p /opt/check_multi
```

```
sudo cp -a . /opt/check_multi/
```

```
sudo ln -sf /opt/check_multi/bin/check_multi /usr/local/bin/check_multi
```

```
sudo chmod 0755 /usr/local/bin/check_multi
```

```
check_multi --version
```

---

3. Post-Install Verification

```
check_multi --version
```

```
check_multi --help
```

```
check_multi --dry-run platform examples/devices.txt
```

```
check_multi --dry-run --excel platform examples/devices.txt
```

---

4. SSH Key Setup (Strongly Recommended)

```
ssh-keygen -t ed25519 -f ~/.ssh/check_multi -C "check_multi-audit"
```

```
ssh-copy-id -i ~/.ssh/check_multi.pub admin@f5-lab-01.example.com
```

```
ssh -i ~/.ssh/check_multi admin@f5-lab-01.example.com "tmsh show sys version"
```

---

5. Common Installation Issues

Symptom	Likely Cause	Fix
`yq: command not found`	yq not installed	Install yq (see above)
Excel report fails	openpyxl missing	`pip install openpyxl`
`Permission denied` on bin/check_multi	Not executable	`chmod 0755 bin/check_multi`
`Check module not found`	Wrong working directory	Run from the repo root or set PATH
Host key warnings	Expected in lab profile	Use `-e prod` or set strict checking

---

6. Uninstall

```
# User-local
rm -rf ~/opt/check_multi
rm -f ~/bin/check_multi

# System-wide
sudo rm -rf /opt/check_multi
sudo rm -f /usr/local/bin/check_multi
```

4. Usage

Usage

`check_multi` runs a single **check** against every device in an **inventory file**, in parallel, and writes structured results.

```
check_multi [OPTIONS] <check_name> <inventory_file>
```

Options

Option	Description	Default
`-h, --help`	Show help and exit	–
`-d, --dry-run`	Never open SSH connections; print what would run	off
`-v, --verbose`	Debug logging	off
`-t, --threads <N>`	Maximum parallel SSH sessions (positive integer)	`12` (or config)
`-o, --output-dir <DIR>`	Results directory	`results/<ts>_<check>`
`-e, --env <name>`	Environment profile (`config/environments/<name>.yaml`)	`lab`
`-c, --config <FILE>`	Alternate `defaults.yaml`	bundled defaults
`-u, --user <USER>`	SSH username	current user
`-p, --password`	Password auth via `sshpass` (**discouraged**)	off
`-y, --yes`	Assume "yes" for confirmation prompts (unattended runs)	off
`--excel`	Also generate an executive Excel report	off
`--no-color`	Disable coloured output (also honours `NO_COLOR`)	off
`--list-checks`	Print available check modules and exit	–
`--version`	Print version and exit	–

Argument precedence

Configuration is resolved in this order (later wins):

1. Built-in defaults
2. `config/defaults.yaml` (or `--config`)
3. `config/environments/<env>.yaml`
4. **Command-line flags** — an explicit flag such as `-t` always wins.

Safety behaviours

These exist to prevent common operator mistakes:

- **Fast validation.** Thread count, check name, inventory existence and the environment profile are validated *before* any connection is attempted.
- **"Did you mean…?"** A mistyped check name suggests close matches and points to `--list-checks`.
- **High-impact confirmation.** A *real* (non-dry-run) execution against the `prod` profile or a large inventory (≥ 50 devices) prompts for confirmation. Pass `-y/--yes` for unattended runs; on a non-interactive shell the run aborts unless `--yes` is given.
- **Path-safe check names.** Check names are restricted to `[A-Za-z0-9_-]`.

Exit codes

Code	Meaning
`0`	Success — all device checks passed
`1`	Usage or configuration error
`4`	One or more device checks failed

Examples

```
# Safe preview — no SSH, populated summary.json
check_multi --dry-run platform examples/devices.txt

# Lab run, 10 threads, with an Excel report
check_multi -e lab -t 10 --excel platform examples/devices.txt
```

```
# Production certificate audit, unattended

check_multi -e prod --yes certs inventory/prod.txt


# Discover available checks

check_multi --list-checks
```

Output layout

```
results/YYYYMMDD_HHMMSS_<check>/

├── audit/run_*.json      # immutable per-run audit record

├── summary.json          # structured results (devices, counts, rate)

└── Device_Audit_Report_*.xlsx   # only with --excel
```

See Configuration for tuning and Troubleshooting for common issues.

5. Configuration

Configuration

`check_multi` reads YAML configuration when `yq` is available. Without `yq`, sane built-in defaults apply and a warning is logged.

Files

File	Purpose
`config/defaults.yaml`	Base settings for every run
`config/environments/<name>.yaml`	Overrides applied with `-e <name>`

File	Purpose

`defaults.yaml`

```
max_threads: 12

ssh:

  connect_timeout: 8       # seconds to establish a connection

  command_timeout: 45      # seconds a remote command may run

  retries: 2               # additional attempts after the first failure

  backoff_seconds: 3       # delay between retries

  strict_host_key_checking: false

output:

  base_dir: "./results"

  keep_days: 30

logging:

  level: info

audit:

  enabled: true
```

Environment profiles

Profiles overlay only the keys they specify. The bundled `prod` profile hardens defaults:

```
# config/environments/prod.yaml

ssh:

  strict_host_key_checking: true

  retries: 3

  backoff_seconds: 5

max_threads: 20

logging:

  level: warn
```

Create a new profile by dropping `config/environments/<name>.yaml` and selecting it with `-e <name>`.

Environment variables

Variable	Effect
`NO_COLOR`	Disable coloured output (see https://no-color.org/)
`SSHPASS`	Pre-supply the SSH password for `-p` (avoids the prompt)
`ASSUME_YES`	When `true`, auto-approve confirmation prompts (same as `--yes`)
`TMPDIR`	Directory for temporary files (all created mode `0600`)

SSH tunables

The SSH layer honours these values (from config or the environment):

- `SSH_CONNECT_TIMEOUT`, `SSH_COMMAND_TIMEOUT`
- `SSH_RETRIES`, `SSH_BACKOFF`
- `STRICT_HOST_KEY` — when `true`, `StrictHostKeyChecking=yes` is enforced and a real `known_hosts` is used; when `false`, host-key checking is disabled (lab convenience) and a warning is printed.

See Security for the rationale behind the defaults.

6. Check Modules

Check Modules & Roadmap

`check_multi` is plugin-driven: every file in `lib/checks/<name>.sh` that defines a `run_check()` function becomes a selectable check. This page lists the modules that ship today and a prioritised roadmap of proposed additions.

Module contract

```
#!/usr/bin/env bash

# check: <name> – <one-line description>

run_check() {

  local device=$1

  ssh_exec "$device" "tmsh ..." || return 1

}
```

- Return `0` on success, non-zero on failure.
- Use `ssh_exec` (never raw `ssh`) so retries, timeouts, dry-run and quoting are handled centrally.
- Keep remote commands read-only. `check_multi` is an **audit** tool; modules must never mutate device state.

Shipping modules

Module	Purpose	Primary command(s)
`preflight`	Connectivity + version smoke test	`echo OK`, `tmsh show sys version`
`platform`	Version, hardware, provisioning	`tmsh show sys version/hardware`, `list sys provision`

Module	Purpose	Primary command(s)
`umm`	Provisioned modules	`tmsh list sys provision`
`certs`	SSL certificate inventory	`tmsh list sys file ssl-cert`
`keys`	SSL key inventory	`tmsh list sys file ssl-key`
`syncgroup`	Device-group sync status	`tmsh show cm sync-status`
`network`	Interfaces & trunks	`tmsh show net interface/trunk`
`ntp`	Clock sync & NTP config	`tmsh list sys ntp`, `ntpq -pn`
`license`	Licence status & feature modules	`tmsh show sys license`
`diskspace`	Filesystem utilisation	`df -h`, `tmsh show sys disk`
`ha`	Active/standby & failover health	`tmsh show sys failover`

Roadmap — proposed modules

Grouped by theme and roughly ordered by value for a staging pre-check workflow.

Change-readiness (high priority)

- **`ucs`** – Verify a recent UCS backup exists and is restorable (`tmsh show sys ucs`, age of newest archive).
- **`config-sync-drift`** – Detect pending/unsynced config across the device group beyond a simple status string.
- **`cert-expiry`** – Flag certificates expiring within *N* days (parse the `expiration` field rather than dumping the inventory).
- **`ssl-profiles`** – Map cert/key usage to client/server SSL profiles to catch orphaned or shared material before rotation.

Health & capacity

- **`cpu-mem`** – TMM/CPU and memory pressure (`tmsh show sys cpu/memory`).
- **`connection-stats`** – Current vs. licensed connection/throughput limits.
- **`pool-health`** – Pools/members with down or forced-offline nodes (`tmsh show ltm pool`).

- **`virtual-servers`** – Virtual server availability and offline VIPs.

Platform & security

- **`software-images`** – Installed/boot volumes and available images (`tmsh show sys software`).
- **`hotfix`** – Applied hotfixes vs. a desired baseline.
- **`accounts`** – Local admin accounts, auth source, and password policy.
- **`snmp-logging`** – SNMP, syslog and remote logging destinations.
- **`dns-resolvers`** – Configured DNS resolvers and reachability.

Reporting & integration

- **JSON/HTML per-check artefacts** – Structured per-device output for each module (not just pass/fail) so reports can show the captured data.
- **Baseline diffing** – Compare a run against a stored golden baseline and report drift.

Suggesting or contributing a module

Open an issue describing the check and the tmsh command(s) it runs, or follow CONTRIBUTING.md to add one directly. New modules should ship with a matching entry in the CI dry-run smoke test and a short note here.

7. Architecture

Architecture – check_multi 2.0

Design Goals

- Modular, testable, and extensible
- Secure by default (keys preferred, temp files locked down)
- Clear separation of execution vs reporting
- Governance-ready (audit trail, environment profiles)
- Operator-friendly CLI with dry-run and Excel output

High-Level Flow

```
CLI (bin/check_multi)
```

```
    │
    ├─ load config + environment profile
    ├─ load check plugin
    ├─ validate inventory
    ├─ start audit run
    │
    ├─ parallel execution (lib/parallel.sh)
    │     └─ ssh_exec (lib/ssh.sh) → run_check()
    │
    ├─ generate summary.json (lib/summary.sh)
    ├─ optional Excel report (scripts/generate_excel_report.py)
    └─ end audit run
```

Key Modules

Module	Responsibility
`lib/common.sh`	Logging, colours, temp files, JSON escape
`lib/ssh.sh`	SSH with retries, timeouts, dry-run, password warning
`lib/inventory.sh`	Load + validate device lists
`lib/parallel.sh`	Controlled concurrency
`lib/config.sh`	YAML defaults + environment overlays
`lib/audit.sh`	Immutable run records

Module	Responsibility
`lib/summary.sh`	Structured summary.json for reporters
`lib/checks/*.sh`	Plugin check implementations

Plugin Contract

Every check module must define:

```
run_check() {
  local device=$1
  # perform work, return 0 on success, non-zero on failure
}
```

Output Layout

```
results/YYYYMMDD_HHMMSS_<check>/
├── audit/
│   └── run_*.json
├── summary.json
└── Device_Audit_Report_*.xlsx   # when --excel used
```

Extension Points

- Add new checks by dropping a file in `lib/checks/`
- Override behaviour via `config/environments/*.yaml`
- Replace the Excel reporter or add additional reporters that consume `summary.json`

8. Security Model

Security Model – check_multi 2.0

Principles

1. Prefer SSH public-key authentication.
2. Never write credentials to disk.
3. Temporary files are mode 0600 and cleaned on exit.
4. Output directories are mode 0750; logs/results are 0640.
5. Host-key checking is controlled by environment profile.
6. Every run produces an audit trail.

Password Mode

Password authentication via `sshpass` is supported only for emergency / migration use. When enabled (`-p`), a large red warning banner is displayed. The password is held only in the `SSHPASS` environment variable and is never written to a file.

Recommendation: Migrate all devices to key-based auth as soon as practical.

Host Key Checking

- Lab profile: `strict_host_key_checking: false` (convenient for lab rebuilds)
- Prod profile: `strict_host_key_checking: true` (recommended)

Temporary Files

All temporary files are created with `mktemp`, immediately set to mode 0600, tracked, and removed by the EXIT/INT/TERM/HUP trap.

Audit Trail

Every run writes an immutable JSON record under `results/.../audit/` containing: - Run ID, timestamps, operator, host - Check name, inventory path - Success / failure counts - Dry-run flag

Hardening Checklist for Production

- [ ] Use SSH keys only (disable `-p`)
- [ ] Set `strict_host_key_checking: true` in prod profile
- [ ] Run from a dedicated jump host with restricted access
- [ ] Keep inventories in a controlled location
- [ ] Review audit logs periodically
- [ ] Pin tool version and review changes before upgrading

9. Troubleshooting

Troubleshooting

Practical fixes for the issues operators hit most often. Start every investigation with `--dry-run` and `-v/--verbose`.

Diagnostics first

```
# Prove the tool, inventory and check are wired up — no network needed
```

```
check_multi --dry-run -v <check> <inventory>
```

Dry-run validates arguments, loads the inventory and exercises the code path without opening a single SSH connection.

Common issues

"requires a value" / usage error

A flag was given without its argument (e.g. `-t` with nothing after it), or the positional `<check>` / `<inventory>` were omitted. Check `--help`.

"Unknown check: …"

The check name is misspelled or the module isn't installed. Run `check_multi --list-checks`; the error also suggests near matches.

"--threads must be a positive integer"

`-t` was given a non-numeric or zero value. Note that an explicit `-t` always overrides `max_threads` from YAML.

"Confirmation required but stdin is not a TTY"

A real run against `prod` (or a large inventory) needs confirmation. In cron/CI, add `-y/--yes` (or set `ASSUME_YES=true`).

SSH failures / timeouts

- Confirm reachability: `ssh <user>@<device>` by hand.
- The tool retries per `ssh.retries`/`ssh.backoff_seconds`; increase them for flaky links.
- Increase `ssh.connect_timeout` / `ssh.command_timeout` for slow devices.
- Prefer **SSH keys**. Password mode (`-p`) is for emergencies only.

Host key verification failed

The `prod` profile enforces `StrictHostKeyChecking=yes`. Add the device to your `known_hosts`, or (lab only) use a profile with `strict_host_key_checking: false`.

YAML config seems ignored

`yq` (mikefarah) isn't installed, so built-in defaults are used — you'll see a warning. Install `yq` to enable `config/*.yaml`.

Excel report not produced

`--excel` needs `python3` and `openpyxl` (`pip install openpyxl`). Without them a warning is logged and the run still succeeds; `summary.json` is always written.

Devices skipped as "invalid inventory entry"

Entries are rejected if they contain whitespace or shell metacharacters. One host per line; use `#` for comments (inline comments are supported).

Where to look

- `results/<run>/summary.json` — machine-readable results and counts.
- `results/<run>/audit/run_*.json` — who ran what, when, and the outcome.
- Re-run with `-v` for per-attempt SSH debug logging on stderr.

Still stuck? Open an issue with the `--dry-run -v` output (redact hostnames).

10. FAQ

FAQ

What is `check_multi`? A modular, parallel, SSH-based audit framework for network appliances (primarily F5 BIG-IP). It runs read-only checks across an inventory and produces structured JSON plus optional Excel reports.

Does it change device configuration? No. `check_multi` is strictly an **audit** tool. Every check runs read-only commands. Modules that mutate state are not accepted (see Modules).

Which platforms are supported? Any SSH-reachable host. The bundled checks target F5 BIG-IP (`tmsh`), but the framework is device-agnostic — write a module with the commands you need.

What are the requirements? Bash 4+, OpenSSH. Optional: `yq` (YAML config), `python3` + `openpyxl` (Excel), `sshpass` (password auth). See Installation.

How do I add a new check? Drop `lib/checks/<name>.sh` defining `run_check()`. It appears automatically in `--list-checks`, tab-completion and the CI smoke test. See Modules and CONTRIBUTING.

How does parallelism work? Up to `--threads` checks run concurrently; the tool throttles new jobs until a slot frees. Tune per environment via `max_threads`.

Is password authentication safe? Prefer SSH keys. Password mode (`-p`) uses `sshpass`, prints a prominent warning, and keeps the secret only in the `SSHPASS` environment variable — never on disk. See Security.

Why was my production run blocked? Real runs against `prod` or large inventories require confirmation to prevent accidents. Use `-y/--yes` for unattended execution.

Where do results go? `results/<timestamp>_<check>/` — containing `summary.json`, an `audit/` record, and an Excel report when `--excel` is used.

How do I run it unattended (cron/CI)? Add `-y`, prefer key auth, and pin the environment profile, e.g. `check_multi -y -e prod certs inventory/prod.txt`.

Can I disable colour? Yes — `--no-color` or set `NO_COLOR=1`. Colour is auto-disabled when output is not a terminal.

11. bigip-precheck — Architecture

bigip-precheck — Architecture

`bigip-precheck` is the Python, iControl REST–driven upgrade-readiness validator that complements the Bash `check_multi` tool. This document describes the phase-A design; later phases (SNMP cross-checks, LTM/GTM object checks, richer reports) slot into the same seams.

Design principles

1. **Read-only.** Nothing mutates a device. There is no flag that changes device state in this release.
2. **Never a silent PASS.** Any check that cannot *positively* confirm health degrades to `WARN`/`FAIL`. Unexpected exceptions in a checker become a `FAIL`, not a dropped result (`checkers.base.timed`).
3. **Auditable.** Every run has a session ID and an append-only, redacted audit trail; the JSON report gets a SHA-256 sidecar.
4. **Testable without a lab.** Checkers depend on a `RestClient` *protocol*, so the whole pipeline runs against synthesized fixtures.

Layers

```
CLI (Typer)
  └─ load_inventory ──────────────► config/ (Pydantic models, secret-free)
  └─ build_default_registry ──────► checkers/ (system, ha; LTM/GTM/SNMP later)
  └─ Orchestrator
       ├─ group_devices ──────────► standby-before-active ordering per HA group
       ├─ RestClient (per device) ► clients/rest.py (token, retry, version probe)
       └─ Checker.run(RunContext) ► CheckResult[]
  └─ Gate ────────────────────────► per-device + overall GO / NO-GO
  └─ reporting/ ──────────────────► Rich console + schema-versioned JSON
```

```
└─ logging/ ──────────────────────► audit JSONL + human log + integrity hash
```

Key abstractions

Type	Responsibility
`CheckResult`	Immutable observation: status, severity, summary, redacted evidence, source.
`Checker` (ABC)	Declares metadata (`name`, `severity`, `applies_to`, `depends_on`) and `run()`.
`Registry`	Registration, profile filtering, and topological ordering by dependency.
`RunContext`	Per-device bundle: device, client, thresholds, detected roles, session id.
`Orchestrator`	Groups/orders devices, runs checkers, skips dependents of failed checks.
`Gate`	Aggregates results into GO / NO-GO (default: any FAIL blocks; `--strict` adds HIGH+ WARN).

Ordering guarantees

- **Standby before active.** Devices sharing an `ha:<group>` tag run sequentially, standby member first (`orchestrator.group_devices`). Distinct groups and standalone devices run concurrently up to `settings.max_workers`.
- **Dependency-aware skips.** A checker whose `depends_on` target FAILed on a device is recorded as an explicit `SKIP`, never silently omitted.

Secrets

Secrets never enter the config models. A device names a `credential`; the value is resolved at run time from `BIGIP_<CRED>_USERNAME` / `_PASSWORD` / `_TOKEN` (with `BIGIP_USERNAME` / `BIGIP_PASSWORD` fallback). All log/report output passes through `redaction.redact` first.

Exit codes

`0` GO · `2` NO-GO · `3` config error. `--ci` makes runs non-interactive and machine-readable; `--json` prints the report to stdout.

Roadmap seams

- **PR B** — `checkers/ltm.py`, `checkers/gtm.py`; role auto-detection over REST; pre/post object-state snapshots.
- **PR C** — `clients/snmp.py` + `checkers/snmp_crosscheck.py`; REST ↔ SNMP discrepancy gating.
- **PR D** — HTML/Markdown reports; report signing; TMSH `load sys config verify` escape hatch.

12. bigip-precheck — Testing

bigip-precheck — Testing strategy

The suite runs entirely offline: no BIG-IP, no network. It lives under `tests/python/` and is driven by `pytest`.

```
pip install -e '.[dev]'

pytest                                  # run the suite

pytest --cov=bigip_precheck             # with coverage

ruff check src tests                    # lint

mypy                                    # strict type check
```

Layers under test

Layer	How it is tested
Checkers (`system`, `ha`)	Unit tests with a `FakeRestClient` fed synthesized iControl payloads.
REST client	`httpx.MockTransport` scripts login / 401-refresh / 5xx-retry / version probe — no sockets.
Registry / gate / orchestrator	Dependency ordering, GO/NO-GO policy, standby-first grouping, dependent-skip.
Config	Inventory validation, duplicate/unknown-field rejection, credential resolution.

Layer	How it is tested
Logging	Audit JSONL redaction; report SHA-256 round-trip and tamper detection.
CLI	`typer.testing.CliRunner`: exit codes, report + digest artefacts, JSON output.

The "never a silent PASS" bias, tested explicitly

`test_checkers_system.py::test_version_client_error_fails_not_passes` asserts that an unreadable device produces a `FAIL`, not a `PASS`. New checkers should add the equivalent negative test: when the device cannot be read, the result must not be `PASS`.

Fixtures

`tests/python/fixtures/icontrol.py` holds hand-built payloads that mirror the shapes TMOS 15.1–17.x return (collections with `items`, stats with the `entries → nestedStats → entries → description` envelope). They are synthesized from public F5 schemas so the suite needs no lab.

Replacing fixtures with real captures: sanitize a real `GET /mgmt/tm/...` response (scrub hostnames, keys, tokens), drop it into `icontrol.py` under the same constant name, and the consuming tests continue to pass unchanged. Keep one healthy and one unhealthy variant per check so both the GO and NO-GO paths stay covered.

Scenario matrix (phase A)

Standalone (sync = Standalone → INFO), HA standby, HA active, expired license, config-sync changes-pending, single boot volume, install-in-progress, unreadable/timed-out REST. LTM/GTM object scenarios and the SNMP-down and mixed-version cases arrive with PRs B and C.

13. bigip-precheck — README

bigip-precheck

Read-only pre-upgrade / change-window readiness validator for F5 BIG-IP (LTM & GTM).

`bigip-precheck` is the Python companion to the Bash `check_multi` tool in this repository. Where

`check_multi` runs fast SSH/tmsh sweeps, `bigip-precheck` is a deeper, **iControl REST**–driven validator that answers one question before a maintenance window: **is this device safe to upgrade — GO or NO-GO?**

It is **read-only**: it never mutates a device. Every run is fully audit-logged, and the design bias is that a check which cannot confirm health degrades to `WARN`/`FAIL` — never a silent `PASS`.

> Status: **phase A (alpha)** — System + HA checks over iControl REST. LTM/GTM checks and the SNMP cross-check layer land in later phases (see the roadmap).

Install

```
pip install -e '.[dev]'      # from the repository root
```

Quick start

```
# Validate an inventory file offline (no device contact):

bigip-precheck validate-config examples/bigip-precheck/inventory.yaml


# List the available checks:

bigip-precheck list-checks


# Supply credentials via the environment (never stored in the inventory):

export BIGIP_DEFAULT_USERNAME=admin

export BIGIP_DEFAULT_PASSWORD='...'


# Run the "full" profile and print a GO/NO-GO dashboard:

bigip-precheck run examples/bigip-precheck/inventory.yaml --profile full


# CI mode: non-interactive, JSON report to stdout, non-zero exit on NO-GO:
```

```
bigip-precheck run inventory.yaml --profile full --ci --json
```

Exit codes

Code	Meaning
`0`	GO — every device is clear to proceed.
`2`	NO-GO — at least one device has a blocking result.
`3`	Config error — the run could not start.

Checks (phase A)

Check	Severity	Source	Purpose
`system.version`	HIGH	REST	Report running TMOS version/build.
`system.license`	CRITICAL	REST	Licensed + service-check date valid (K7727).
`system.provisioning`	MEDIUM	REST	Provisioned modules and levels.
`system.boot-volumes`	HIGH	REST	A free install target exists; nothing mid-install.
`ha.failover-status`	HIGH	REST	Failover role (Active/Standby) + traffic-group health.
`ha.sync-status`	HIGH	REST	Config-sync is In Sync (depends on failover-status).

Configuration

See `examples/bigip-precheck/inventory.yaml`. Devices reference a `credential` by name; the secret is resolved at runtime from `BIGIP_<CRED>_USERNAME` / `BIGIP_<CRED>_PASSWORD` (or `BIGIP_<CRED>_TOKEN`), with `BIGIP_USERNAME` / `BIGIP_PASSWORD` as a fallback. HA members that share an `ha:<group>` tag are evaluated **standby before active**.

Roadmap

- **PR B** — LTM + GTM object-state checks with pre/post snapshots.
- **PR C** — SNMP layer and REST ↔ SNMP cross-checks (catch single-source blind spots).

- **PR D** — HTML/Markdown reports, report signing, TMSH `load sys config verify`.

See `docs/bigip-precheck/TESTING.md` for the test strategy and how synthesized fixtures are replaced with real captures.